

Playwright E2E Test Plan — Complete IIDM Application

For: QA Engineer **Application:** IIDM Buzzworld (React frontend → Lumen API → PostgreSQL) **Framework:** Playwright (Node.js) **Existing:** Login + 11 module navigation tests (78.6% pass rate) **Goal:** Full application coverage — every user workflow automated

Reference docs:

- Frontend: docs/05_frontend_workflow_catalog.md (all workflows traced)
- Frontend: docs/01_frontend_architecture_map.md (105 routes, 262 endpoints)
- Frontend: docs/04_frontend_risk_hotspots.md (high-risk areas)
- API: docs/05_workflow_catalog.md (backend workflow details)

Test Framework Setup

Project Structure

e2e/	
├─ playwright.config.ts	← Config (base URL, timeouts, browsers)
├─ package.json	
├─ .env.test	← Test credentials (git-ignored)
├─ helpers/	
├─ auth.ts	← Login helper (reuse across all tests)
├─ navigation.ts	← Navigate to module helper
├─ grid.ts	← AG Grid interaction helpers
├─ form.ts	← Form fill/submit helpers
├─ assertions.ts	← Common assertions (toast, modal, grid data)
└─ test-data.ts	← Test data generators (unique names, UUIDs)
├─ fixtures/	
├─ users.ts	← User credentials per role (admin, sales, customer)
└─ test-data.ts	← Reusable test data (org names, products, etc.)
├─ tests/	
├─ 01-auth/	← Login, logout, permissions
├─ 02-quotes/	← Full quote lifecycle
├─ 03-repairs/	← Full repair lifecycle
├─ 04-jobs/	← Jobs workflows
├─ 05-orders/	← Sales order workflows
├─ 06-parts-purchase/	← Parts purchase workflows
├─ 07-pricing/	← Pricing, discount codes, special pricing
└─ 08-inventory/	← Inventory management

— 09-organizations/	← Orgs, contacts, addresses
— 10-admin/	← All 23 admin CRUD pages
— 11-dashboard/	← Dashboard widgets, metrics
— 12-reports/	← Past-due invoices, point of sales
— 13-cross-module/	← Workflows spanning multiple modules
— reports/	← HTML test reports

Helper Files to Create First

Before writing any tests, build these helpers — they save massive time:

auth.ts — Login helper (used by EVERY test)

typescript

```
// Login once, save auth state, reuse across tests
// Playwright's storageState feature avoids re-login per test

export async function login(page, username, password) {
  await page.goto('/');
  // Handle SSO redirect
  // Fill credentials
  // Wait for redirect to quotes page
  // Save storage state for reuse
}

export async function loginAsAdmin(page) { /* admin creds */ }
export async function loginAsSales(page) { /* sales creds */ }
export async function loginAsCustomer(page) { /* customer creds */ }
```

grid.ts — AG Grid helpers (19 grids in the app)

typescript

```

// Interact with PiServerGrid2 grids consistently

export async function waitForGridLoad(page) {
  await page.waitForSelector('.ag-body-viewport .ag-row');
}

export async function getGridRowCount(page) {
  return await page.locator('.ag-row').count();
}

export async function clickGridRow(page, rowIndex) {
  await page.locator(`.ag-row[row-index="${rowIndex}"]`).click();
}

export async function getGridCellValue(page, rowIndex, colId) {
  return await page.locator(
    `.ag-row[row-index="${rowIndex}"] [col-id="${colId}"]`
  ).textContent();
}

export async function sortGridColumn(page, colId) {
  await page.locator(`.ag-header-cell[col-id="${colId}"]`).click();
}

export async function filterGrid(page, colId, value) {
  // Open filter, type value, apply
}

export async function exportGrid(page, format) {
  // Click export button, select format
}

```

form.ts — Form helpers

typescript

```
export async function fillInput(page, label, value) {
  await page.getByLabel(label).fill(value);
}

export async function selectDropdown(page, label, option) {
  await page.getByLabel(label).click();
  await page.getByRole('option', { name: option }).click();
}

export async function fillDatePicker(page, label, date) {
  // Handle react-datepicker
}

export async function submitForm(page) {
  await page.getByRole('button', { name: /save|submit|create/i }).click();
}

export async function expectToast(page, message) {
  await expect(page.locator('.toast, .snackbar, [role="alert"]'))
    .toContainText(message);
}
```

test-data.ts — Unique test data

typescript

```
// Generate unique data to avoid conflicts between test runs

export function uniqueName(prefix: string) {
  return `${prefix}_TEST_${Date.now()}`;
}

export function testOrg() {
  return {
    name: uniqueName('ORG'),
    phone: '555-0100',
    email: `test_${Date.now()}@test.com`,
  };
}

export function testQuote() {
  return {
    // quote-specific test data
  };
}
```

Test Priority & Execution Order

Write and run tests in this order — most critical workflows first:

Priority	Module	Tests	Why First
P0	Auth & Login	8	Everything depends on auth working
P1	Quotes	25	Revenue-critical — most used module
P1	Repairs	20	Second most used module
P2	Orders	12	Sales order creation from quotes
P2	Parts Purchase	12	Procurement workflow
P2	Pricing	15	Pricing rules affect all quotes
P3	Organizations	10	CRM data, contacts
P3	Admin	23	One CRUD test per admin page
P3	Jobs	8	Job tracking
P3	Inventory	6	Inventory queries
P4	Dashboard	8	Data display, less interaction
P4	Reports	5	Report generation
P4	Cross-Module	10	Workflows spanning modules
TOTAL		~162	

Detailed Test Cases by Module

01 — Authentication & Permissions (8 tests)

File: tests/01-auth/login.spec.ts

AUTH-001: Successful SSO login with valid credentials

- Go to app URL
- SSO redirect happens
- Enter username/password
- Redirected to /quote_for_parts (sales) or /pricing (admin)
- User name visible in header

AUTH-002: Login with invalid credentials

- Enter wrong password
- Error message shown
- Not redirected into app

AUTH-003: Session expiry / 401 handling

- Login successfully
- Clear token from localStorage
- Navigate to any page
- Redirected to login

AUTH-004: Admin user sees admin menu items

- Login as admin
- Verify: Admin, Pricing, all menu items visible
- Navigate to /admin → page loads

AUTH-005: Sales user sees sales menu items

- Login as sales user
- Verify: Quotes, Repairs, Jobs visible
- Admin menu NOT visible (or access-denied if navigated directly)

AUTH-006: Customer portal user sees limited menu

- Login as customer
- Verify: only customer-facing pages visible
- Direct URL to /admin → access denied

AUTH-007: Logout clears session

- Login → verify logged in
- Click logout
- Redirected to login page
- Back button doesn't return to app
- localStorage token cleared

AUTH-008: Role-based landing page

- Admin login → lands on /pricing
- Sales login → lands on /quote_for_parts

02 — Quotes Module (25 tests)

File: tests/02-quotes/quote-list.spec.ts

QUOTE-001: Quote list loads with data

- Navigate to /all_quotes
- Grid loads with rows
- Columns visible: quote #, customer, status, amount, date
- Pagination works (next/prev)

QUOTE-002: Filter quotes by status

- Select status filter (e.g., "Draft")
- Grid updates to show only draft quotes
- Clear filter → all quotes return

QUOTE-003: Search quotes

- Type in search box
- Grid filters by search term
- Clear search → all quotes return

QUOTE-004: Sort quotes by column

- Click column header (e.g., Date)
- Rows reorder
- Click again → reverse order

QUOTE-005: Navigate to quote detail

- Click a row in the grid
- Redirected to /quote/{id}
- Quote detail page loads with correct data

File: tests/02-quotes/quote-create.spec.ts

QUOTE-006: Create new quote

- Click "Add Quote" button
- Side drawer / form opens
- Select organization from dropdown
- Fill required fields
- Click Save
- Toast: "Quote created successfully"
- New quote appears in grid

QUOTE-007: Create quote with missing required fields

- Click "Add Quote"
- Leave organization empty
- Click Save
- Validation error shown for required fields
- Quote NOT created

QUOTE-008: Add items to quote

- Open existing quote detail
- Click "Add Item"
- Search/select a product
- Set quantity and price
- Save item
- Item appears in items list
- Quote total updates

QUOTE-009: Edit quote item

- Open quote with items
- Click edit on an item
- Change quantity
- Save
- Total recalculates

QUOTE-010: Delete quote item

- Open quote with items
- Click delete on an item
- Confirm deletion
- Item removed
- Total recalculates

QUOTE-011: Add quote options

- Open quote detail
- Navigate to Options tab
- Add an option
- Save
- Option visible in list

File: tests/02-quotes/quote-lifecycle.spec.ts

QUOTE-012: Submit quote for internal approval

- Open draft quote
- Click "Submit for Approval"
- Status changes to "Pending Approval"
- Approval notification sent (verify status change, not email)

QUOTE-013: Approve quote

- Login as approver
- Open pending quote
- Click "Approve"

- Status changes to "Approved"

QUOTE-014: Reject quote

- Login as approver
- Open pending quote
- Click "Reject" with reason
- Status changes to "Rejected"
- Reason visible

QUOTE-015: Send quote to customer

- Open approved quote
- Click "Send to Customer"
- Status changes to "Sent to Customer"

QUOTE-016: Close quote (won)

- Open sent quote
- Click "Close" → select "Won"
- Status changes to "Won"

QUOTE-017: Close quote (lost)

- Open sent quote
- Click "Close" → select "Lost"
- Enter lost reason
- Status changes to "Lost"

QUOTE-018: Reopen closed quote

- Open won/lost quote
- Click "Reopen"
- Status changes back to "Draft"

QUOTE-019: Clone quote

- Open any quote
- Click "Clone"
- New quote created with same items
- New quote number generated
- Status is "Draft"

QUOTE-020: Revise quote

- Open sent quote
- Click "Revise"
- New version created
- Old version archived

QUOTE-021: Print quote PDF

- Open quote detail
- Click "Print PDF"
- PDF generates (verify download or preview)

QUOTE-022: Import quote items from Excel

- Open quote
- Click "Import Items"
- Upload Excel file
- Items added to quote

QUOTE-023: Export quote to Excel

- Open quote list
- Click Export
- File downloads

File: tests/02-quotes/quote-navigation.spec.ts

QUOTE-024: All quote filter views work

- /quote_for_parts → loads
- /quote_for_repair → loads
- /all_quotes → loads
- /quoted_by → loads
- /waiting_on_me → loads
- /system_quotes → loads
- /quote_expired → loads
- /quote_archived → loads

QUOTE-025: Quote detail tabs all load

- Open a quote
- Click each tab: Items, Options, Notes, Attachments, Logs, Approvals
- Each tab content loads without error

03 — Repairs Module (20 tests)

File: tests/03-repairs/repair-list.spec.ts

REPAIR-001: Repair list loads

REPAIR-002: Filter repairs by status

REPAIR-003: All repair status views load

- /repair-request, /shipped, /billing, /check_in, /evaluation,

/qc, /repair_in_progress, /waiting_on_parts, /my-repairs,
 /pending_quote, /added_to_quote, /receiving
→ Each loads with correct grid data

File: tests/03-repairs/repair-create.spec.ts

REPAIR-004: Create RMA request

- Click "New Repair Request"
- Select organization
- Fill required fields
- Save
- RMA created with status "Repair Request"

REPAIR-005: Create RMA with missing fields → validation error

File: tests/03-repairs/repair-lifecycle.spec.ts

REPAIR-006: Check-in repair items

- Open RMA in "Repair Request" status
- Click "Check In"
- Add serial numbers / item details
- Status → "Checked In"

REPAIR-007: Evaluate repair items

- Open checked-in repair
- Add evaluation notes
- Set repair/replace/return decision per item
- Status → "Evaluation"

REPAIR-008: QC inspection

- Open evaluated repair
- Complete QC checklist
- Pass/fail items
- Status → "QC Complete"

REPAIR-009: Repair in progress

- Open QC'd repair
- Update work status
- Add work notes
- Status → "In Progress"

REPAIR-010: Waiting on parts

- Mark repair as waiting for parts
- Link to parts purchase

REPAIR-011: Billing

- Open completed repair
- Generate billing
- Status → "Billing"

REPAIR-012: Ship repaired items

- Open billed repair
- Enter shipping details (carrier, tracking)
- Status → "Shipped"

REPAIR-013: Add internal notes to repair

REPAIR-014: Add attachments to repair

REPAIR-015: View repair logs/history

REPAIR-016: Generate RMA PDF

REPAIR-017: Assign repair to technician

File: tests/03-repairs/repair-items.spec.ts

REPAIR-018: Add repair item

REPAIR-019: Edit repair item

REPAIR-020: Delete repair item

04 — Jobs Module (8 tests)

File: tests/04-jobs/jobs.spec.ts

JOBS-001: Jobs list loads with data

JOBS-002: Filter jobs by status

JOBS-003: Navigate to job detail

JOBS-004: Job detail tabs load (items, checklist, time tracking)

JOBS-005: Add job checklist item

JOBS-006: Log time against job

JOBS-007: Update job status

JOBS-008: Search jobs

05 — Sales Orders (12 tests)

File: tests/05-orders/orders.spec.ts

ORDER-001: Orders list loads

ORDER-002: Filter/sort orders

ORDER-003: Navigate to order detail

File: tests/05-orders/create-so.spec.ts

ORDER-004: Create sales order from won quote

- Open won quote
- Click "Create Sales Order"
- SO form pre-fills from quote data
- Verify items copied
- Submit SO
- SO created

ORDER-005: Create SO with modified items

- Create SO from quote
- Remove an item
- Change quantity on another
- Submit

ORDER-006: SO validation (missing required fields)

ORDER-007: Insert SO to SYSPRO

- Create SO
- Click "Insert to SYSPRO"
- Success message (or "SYSPRO sync skipped" in QA)
- Status updates

ORDER-008: Order detail tabs load

ORDER-009: Add items to existing order

ORDER-010: Edit order items

ORDER-011: Delete order items

ORDER-012: Export orders

06 — Parts Purchase (12 tests)

File: tests/06-parts-purchase/parts.spec.ts

PARTS-001: Parts purchase list loads

PARTS-002: All status views load (/ordered, /requested, /partially_received, etc.)

PARTS-003: Create parts purchase request

PARTS-004: Create PO from request

PARTS-005: Add items to PO

PARTS-006: Edit PO items

PARTS-007: Delete PO items

PARTS-008: Receive items (partial)

PARTS-009: Receive items (complete)

PARTS-010: Cancel parts purchase

PARTS-011: Approve parts purchase

PARTS-012: Navigate to detail view

07 — Pricing (15 tests)

File: tests/07-pricing/pricing.spec.ts

PRICING-001: Pricing page loads with vendor grid

PRICING-002: Select vendor → pricing data loads

PRICING-003: Filter pricing by branch

PRICING-004: Export pricing data

File: tests/07-pricing/discount-codes.spec.ts

PRICING-005: Discount codes page loads

PRICING-006: Add discount code

PRICING-007: Edit discount code

PRICING-008: Activate discount code

PRICING-009: Deactivate discount code

File: tests/07-pricing/special-pricing.spec.ts

PRICING-010: Special pricing list loads

PRICING-011: Create special pricing card

PRICING-012: Add products to pricing card

PRICING-013: Activate special pricing
PRICING-014: Import pricing from Excel
PRICING-015: Pricing rule configurator (create + edit rules)

08 — Inventory (6 tests)

File: tests/08-inventory/inventory.spec.ts

INV-001: Inventory list loads
INV-002: Search inventory by stock code
INV-003: Filter inventory
INV-004: View inventory detail
INV-005: Export inventory
INV-006: Inventory query (SYSPRO) – if available

09 — Organizations & Contacts (10 tests)

File: tests/09-organizations/orgs.spec.ts

ORG-001: Organization list loads
ORG-002: Create new organization
ORG-003: Edit organization
ORG-004: Search organizations
ORG-005: View organization detail

File: tests/09-organizations/contacts.spec.ts

ORG-006: Contacts list loads
ORG-007: Create contact under organization
ORG-008: Edit contact
ORG-009: Add address to organization
ORG-010: Sync organization to Dynamics (or verify "sync" button exists)

10 — Admin CRUD (23 tests — one per admin page)

File: tests/10-admin/admin-crud.spec.ts

For EACH admin page, test the same CRUD pattern:

1. List loads with grid data
2. Add new record → success toast → appears in grid
3. Edit existing record → save → updated in grid
4. Delete record → confirm → removed from grid (if delete is available)

ADMIN-001: Account Types

ADMIN-002: Classifications

ADMIN-003: Industry

ADMIN-004: Sales Potential

ADMIN-005: Contact Types

ADMIN-006: Shipping Types

ADMIN-007: FOB Types

ADMIN-008: Branches

ADMIN-009: Regions

ADMIN-010: Territory

ADMIN-011: Zipcodes

ADMIN-012: Warehouses

ADMIN-013: Vendors

ADMIN-014: Users

ADMIN-015: User Roles

ADMIN-016: Permissions

ADMIN-017: QC Control

ADMIN-018: Products

ADMIN-019: Manufacturer Branches

ADMIN-020: Manufacturer Discount

ADMIN-021: Special Pricing Cards

ADMIN-022: Pricing Rules

ADMIN-023: Email Templates

TIP: Use a data-driven pattern – one test function with parameterized admin pages, not 23 separate test functions:

```
const adminPages = [  
  { name: 'Account Types', url: '/account-type', addButton: 'Add' },  
  { name: 'Branches', url: '/branches', addButton: 'Add Branch' },  
  // ...  
];
```

```
for (const page of adminPages) {  
  test(`Admin CRUD: ${page.name}`, async ({ page: p }) => {  
    await p.goto(page.url);  
    await waitForGridLoad(p);
```

```
// ... CRUD assertions
});
}
```

11 — Dashboard (8 tests)

File: tests/11-dashboard/dashboard.spec.ts

DASH-001: Sales dashboard loads all widgets
DASH-002: Company metrics page loads
DASH-003: Company metrics detail drill-down
DASH-004: Dashboard date range selector works
DASH-005: Dashboard data refreshes on sync
DASH-006: Appointments widget loads
DASH-007: Opportunities widget loads
DASH-008: Needs Your Attention widget loads

12 — Reports (5 tests)

File: tests/12-reports/reports.spec.ts

REPORT-001: Past due invoices page loads
REPORT-002: Past due invoices filter works
REPORT-003: Point of sales report loads
REPORT-004: Export report data
REPORT-005: Inventory report loads

13 — Cross-Module Workflows (10 tests)

File: tests/13-cross-module/cross-module.spec.ts

CROSS-001: Quote → Sales Order → SYSPRO
→ Create quote → add items → approve → won → create SO → insert to SYSPRO
→ Verify entire chain works end-to-end

CROSS-002: Repair → Quote → Approval
→ Create RMA → evaluate → generate quote for repair → approve quote

CROSS-003: Organization → Quote → Contact

- Create org → add contact → create quote for that org
- Verify org and contact visible in quote

CROSS-004: Parts Purchase → Inventory

- Create PO → receive items → verify inventory updated

CROSS-005: Pricing → Quote Items

- Set special pricing → create quote → add item → verify price applied

CROSS-006: User creation → Permission assignment → Access verification

- Create user → assign role → login as new user → verify access

CROSS-007: Bulk operations

- Select multiple quotes → bulk action (export, status change)

CROSS-008: File upload → Attachment → Download

- Upload file to quote → verify in attachments → download

CROSS-009: Search across modules

- Global search → verify results from multiple modules

CROSS-010: Grid state persistence

- Set filters and column order → navigate away → come back
- Verify grid state restored from localStorage

Test Execution Strategy

Daily (Smoke Tests) — ~5 minutes

```
bash
```

```
npx playwright test --grep "@smoke" --reporter=html
```

Tag critical tests with `@smoke`:

- AUTH-001 (login)
- QUOTE-001 (quote list loads)
- REPAIR-001 (repair list loads)
- ORDER-001 (orders list loads)

- DASH-001 (dashboard loads)

PR Verification — ~15 minutes

```
bash  
  
npx playwright test --grep "@regression" --reporter=html
```

Tag all tests except cross-module as `@regression`.

Full Suite (Sprint End) — ~30 minutes

```
bash  
  
npx playwright test --reporter=html
```

All tests including cross-module workflows.

CI/CD Integration

```
yaml
```

```
# .github/workflows/e2e-tests.yml
name: E2E Tests
on:
  pull_request:
    branches: [main, stage]
  push:
    branches: [main]

jobs:
  e2e:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npx playwright test --grep "@smoke"
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```

Test Writing Rules for QA

1. **Every test must be independent** — can run alone, in any order
2. **Create test data, don't depend on existing data** — use `uniqueName()` helper
3. **Clean up after tests** — delete created records or use test-specific data that won't interfere
4. **Use helpers for common actions** — don't repeat login/grid/form code
5. **Wait for elements, don't use fixed delays** — `waitForSelector`, `waitForResponse`, never `page.waitForTimeout(3000)`
6. **Take screenshots on failure** — Playwright does this automatically with default config
7. **Tag tests with @smoke, @regression, @critical** — enables selective execution
8. **One assertion per logical check** — multiple expects are fine, but each should test one thing
9. **Test user behavior, not implementation** — click buttons by text, not by CSS class

10. **Handle SYSPRO-dependent features** — if SYSPRO_ENABLED=false in QA, verify the "skipped" message instead of SYSPRO result
-

Playwright Config

```
typescript

// playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  retries: 1,
  workers: 1, // sequential - avoids test data conflicts

  use: {
    baseURL: process.env.BASE_URL || 'http://192.168.x.x:3000',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry',
  },

  reporter: [
    ['html', { open: 'never' }],
    ['list'],
  ],

  projects: [
    {
      name: 'chromium',
      use: { browserName: 'chromium' },
    },
    // Add later:
    // { name: 'firefox', use: { browserName: 'firefox' } },
    // { name: 'webkit', use: { browserName: 'webkit' } },
  ],
});
```

Reporting Template

After each test suite run, QA fills this:

markdown

E2E Test Report – Sprint XX

Date: YYYY-MM-DD
Environment: QA (192.168.x.x)
Browser: Chromium
Execution Time: X minutes

Results

Module	Total	Passed	Failed	Skipped
Auth	8			
Quotes	25			
Repairs	20			
Orders	12			
Parts Purchase	12			
Pricing	15			
Inventory	6			
Organizations	10			
Admin	23			
Dashboard	8			
Reports	5			
Cross-Module	10			
TOTAL	**162**			

Failures

Test ID	Test Name	Error	Screenshot

New Bugs Found

Test ID	Description	Severity	Jira Ticket

SYSPRO-Dependent Tests

- SYSPRO_ENABLED: true / false
- Tests affected: [list]
- Behavior: [pass with real SYSPRO / pass with skip message / fail]

Rollout Schedule

Week	What QA Does	Tests Added	Running Total
1	Setup helpers + Auth + Quotes list/create	~15	15
2	Quote lifecycle + Repairs list/create	~20	35
3	Repair lifecycle + Orders	~20	55
4	Parts Purchase + Pricing	~20	75
5	Organizations + Admin CRUD	~25	100
6	Dashboard + Reports + Inventory	~20	120
7	Cross-module + remaining gaps	~20	140
8	Polish, CI/CD integration, stabilize	~22	162

QA can start immediately with the existing Playwright setup. Week 1 focus: build the helpers (auth, grid, form) and write the first 15 tests. These helpers make all subsequent tests faster to write.